

Average treatment effect for the ACTG data

Ezequiel

2025-03-07

```
# load libraries
library(hdbayes)
library(parallel)
library(matrixStats)
library(MCMCpack)
library(bayestestR)
library(ggplot2)
library(dplyr)
```

Introduction

In this report, we explore the use of distributions in the model space to estimate the average treatment effect (ATE) in a Bayesian framework. The approach leverages historical data to inform prior distributions, incorporates likelihood-based model selection, and applies posterior computations to derive meaningful insights.

We begin by analyzing the ATE with a frequentist approach, using empirical means and generalized linear models to compare the results. First, we compute the empirical ATE (the difference between the empirical means of the treatment and control groups): the average response in the treatment group is 0.0449 and in the control group is 0.0745, resulting in an empirical ATE of -0.0296 . Then, we fit a logistic regression model with the treatment as the only covariate, obtaining an ATE of -0.0296 with standard error of 0.0349, which matches the empirical ATE. Lastly, we fit a logistic regression model with all covariates, obtaining an ATE of -0.0045 with standard error of 0.0337.

To estimate the ATE with Bayesian framework, we begin by defining the model space and establishing prior distributions for model parameters, incorporating historical data through likelihood functions and prior beliefs. Then, we split the dataset into treatment and control groups and compute the posterior distributions for each model within both groups. The weighted combination of these posterior distributions, using Dirichlet-weighted averaging to compute the marginal means and two methods to estimate the ATE, provides a comprehensive view of the treatment effect.

Finally, we present the results, including posterior model probabilities, estimated treatment effects, and visualizations that illustrate the impact of covariates on treatment outcomes.

Distributions on the model space

Let $\mathcal{M}$ be the set of all possible model and

$$p_0(m) = \int L\left(\beta^{(m)} \mid D_0^{(m)}\right) \pi_0\left(\beta^{(m)} \mid d_0\right) d\beta^{(m)},$$
$$\beta^{(m)} \sim N(\mathbf{0}, d_0 \times I_q).$$

where $L(\beta^{(m)} | D_0^{(m)})$ is the likelihood of the historical data $D_0^{(m)}$ under model m , $\pi_0(\beta^{(m)} | d_0)$ is the prior distribution of the regression coefficients, and d_0 is the prior precision of the regression coefficients. We assume that the prior probability of each model is given by

$$p(m) = \frac{p_0(m)}{\sum_{m \in \mathcal{M}} p_0(m)}.$$

Now, we set the joint prior distribution for the regression coefficients and a_0 as

$$\begin{aligned} \pi(\beta^{(m)}, a_0 | D_0^{(m)}) &\propto \frac{L(\beta^{(m)} | D_0^{(m)})^{a_0}}{Z(a_0)} a_0^{\delta_0-1} (1-a_0)^{\lambda_0-1} \pi_0(\beta^{(m)} | c_0), \\ Z(a_0) &= \int L(\beta^{(m)} | D_0^{(m)})^{a_0} \pi_0(\beta^{(m)} | c_0) d\beta^{(m)} \end{aligned}$$

and the posterior distribution of the model m as

$$\begin{aligned} p(m | D^{(m)}) &= \frac{p(D^{(m)} | m) p(m)}{\sum_{m \in \mathcal{M}} p(D^{(m)} | m) p(m)} = \frac{p(D^{(m)} | m) p_0(m)}{\sum_{m \in \mathcal{M}} p(D^{(m)} | m) p_0(m)}, \\ p(D^{(m)} | m) &= \int L(\beta^{(m)} | D^{(m)}) \pi(\beta^{(m)}, a_0 | D_0^{(m)}) d\beta^{(m)} da_0. \end{aligned}$$

Estimation of the average treatment effect

The average treatment effect can be computed through the following steps:

1. Split the data into two groups: control ($D_c^{(m)}$) and treatment ($D_t^{(m)}$).
2. Compute the posterior distribution of each model m for the control and treatment groups, $p(m | D_c^{(m)})$ and $p(m | D_t^{(m)})$, for $m \in \mathcal{M}$.
3. Compute the posterior distribution of the regression coefficients for each model m in the control and treatment groups, $p(\beta^{(m)} | D_c^{(m)})$ and $p(\beta^{(m)} | D_t^{(m)})$, for $m \in \mathcal{M}$, and get $\mu_{am} \approx \sum_{i=1}^n w_i \text{logit}^{-1}(X_{mi}^\top \beta_a^{(m)})$, where n is the number of observations and $\beta_a^{(m)}$ is the regression coefficients, for model m in group $a = c, t$; $(w_1, \dots, w_n) \sim \text{Dirichlet}(1, \dots, 1)$, and X_{mi} is the matrix of covariates.
4. Get $\mu_a = \sum_{m \in \mathcal{M}} p(m | D_a^{(m)}) \mu_{am}$, for $a = c, t$.
5. Compute the average treatment effect as $\mu_t - \mu_c$.

Computations

Posterior samples

To compute p_0 , we use the `glm.npp.lognc` function with $a_0 = 1$. To compute $Z(a_0)$, we use the same function with different values of a_0 . To compute the posterior for the regression coefficients, we use the `glm.npp` function. Lastly, to compute $p(D^{(m)} | m)$, we use the `glm.logml.npp` function.

We create auxiliary functions to compute posterior model probabilities:

```

# function to compute p0
logp0 <- function(formula,
                  ...) {
  hdbayes::glm.npp.lognc(
    formula = formula,
    ...
  )
}

# function to compute Z(a_0)
logncfun <- function(a0,
                    ...){
  hdbayes::glm.npp.lognc(
    a0 = a0,
    ...
  )
}

# function to compute the posterior for regression coefficients
post_beta <- function(formula,
                      c0,
                      data,
                      family,
                      delta0,
                      lambda0,
                      iter_warmup,
                      iter_sampling) {
  # compute Z(a_0) for each value of a_0
  a0.lognc <- lapply(a0_seq,
                    logncfun,
                    formula = formula,
                    family = family,
                    histdata = data[[2]],
                    iter_warmup = iter_warmup,
                    iter_sampling = iter_sampling,
                    chains = 4,
                    refresh = 0,
                    beta.sd = c0
  )
  a0.lognc <- data.frame(do.call(rbind, a0.lognc))
  # compute the posterior for the regression coefficients
  fit <- hdbayes::glm.npp(formula,
                        data = data,
                        family = family,
                        a0.lognc = a0.lognc$a0,
                        lognc = matrix(a0.lognc$lognc, ncol = 1),
                        a0.shape1 = delta0,
                        a0.shape2 = lambda0,
                        iter_warmup = iter_warmup,
                        iter_sampling = iter_sampling,
                        chains = 4,
                        refresh = 0,
                        beta.sd = c0
  )
}

```

```

)
}

# function to compute the posterior model probabilities
post_models <- function(logp0_models, logml_models) {
  logp0_models <- sapply(logp0_models, function(x) x[2])
  logml_models <- unlist(data.frame(do.call(rbind, logml_models))$logml)
  post_model <- exp(logml_models + logp0_models -
                    logSumExp(logml_models + logp0_models))
}

```

Lastly, we create a function to compute the results:

```

# function to compute the results
samples_models <- function(data,
                           outcome,
                           family,
                           a0_seq,
                           c0,
                           d0,
                           delta0,
                           lambda0,
                           iter_warmup,
                           iter_sampling,
                           num_cores = 10) {

  # get data
  current_data <- data[[1]]
  hist_data <- data[[2]]

  # create list of formulas
  covariates <- colnames(current_data)[colnames(current_data) != outcome]
  pset <- powerset(covariates)
  formulas <- lapply(pset, create_formula, outcome = outcome)

  # get covariates from models
  covariates_models <- lapply(pset, get_covariates)

  # define cluster
  cl <- parallel::makeCluster(num_cores)
  on.exit(parallel::stopCluster(cl))
  parallel::clusterExport(cl, varlist = c('a0_seq',
                                           'family',
                                           'data',
                                           'c0',
                                           'd0',
                                           'delta0',
                                           'lambda0',
                                           'iter_warmup',
                                           'iter_sampling',
                                           'logncfun'),
                          envir = environment())

  # compute p0

```

```

logp0_models <- parLapply(
  cl = cl,
  X = formulas,
  fun = logp0,
  family = family,
  histdata = hist_data,
  a0 = 1,
  iter_warmup = iter_warmup,
  iter_sampling = iter_sampling,
  chains = 4,
  refresh = 0,
  beta.sd = d0
)

# compute posterior for regression coefficients
post_betam <- parLapply(
  cl = cl,
  X = formulas,
  fun = post_beta,
  c0 = c0,
  data = data,
  family = family,
  delta0 = delta0,
  lambda0 = lambda0,
  iter_warmup = iter_warmup,
  iter_sampling = iter_sampling
)

# compute logml for models
logml_models <- parLapply(
  cl = cl,
  X = post_betam,
  fun = hdbayes::glm.logml.npp
)

# compute prior model probabilities
prior_models <- exp(sapply(logp0_models, function(x) x[2]) -
  logSumExp(sapply(logp0_models, function(x) x[2])))
)

# compute posterior model probabilities
post_models <- post_models(logp0_models, logml_models)

# create data frame with results
df_post <- data.frame(model = unlist(covariates_models),
  prior_model = prior_models,
  ml = exp(unlist(data.frame(do.call(rbind, logml_models))$logml)),
  post_model = post_models
)

# order data frame
df_post_ord <- df_post[order(df_post$post_model, decreasing = TRUE),]
rownames(df_post_ord) <- 1:nrow(df_post_ord)

```

```

# return results
return(list(logp0_models = logp0_models,
            post_betam = post_betam,
            logml_models = logml_models,
            df_post = df_post,
            df_post_ord = df_post_ord))
}

```

Average treatment effect

First, we normalize the data:

```

# normalize data
age_stats <- with(actg036,
                  c('mean' = mean(age), 'sd' = sd(age)))
cd4_stats <- with(actg036,
                  c('mean' = mean(cd4), 'sd' = sd(cd4)))
actg036$age <- ( actg036$age - age_stats['mean'] ) / age_stats['sd']
actg019$age <- ( actg019$age - age_stats['mean'] ) / age_stats['sd']
actg036$cd4 <- ( actg036$cd4 - cd4_stats['mean'] ) / cd4_stats['sd']
actg019$cd4 <- ( actg019$cd4 - cd4_stats['mean'] ) / cd4_stats['sd']

```

Then, we split the data:

```

# define data
current_data <- actg036
hist_data <- actg019

# split data
current_data_ctrl <- current_data[current_data$treatment == 0,
                                   !(names(current_data) %in% c("treatment"))]
current_data_trt <- current_data[current_data$treatment == 1,
                                   !(names(current_data) %in% c("treatment"))]

```

Now, we set the parameters:

```

beta_pars <- elicit_beta_mean_cv(m0 = 0.5, v0 = 0.005)
a0_seq <- seq(0, 1, length.out = 21)
data_ctrl <- list(current_data_ctrl, hist_data)
data_trt <- list(current_data_trt, hist_data)
family <- binomial(link = "logit")
c0 <- 3^0.5
d0 <- 0.5^0.5
delta0 <- beta_pars$a
lambda0 <- beta_pars$b
iter_warmup <- 1000
iter_sampling <- 2500

```

Finally, we run the model for each group:

```

post_samples_ctrl <- samples_models(data = data_ctrl,
                                   outcome = "outcome",
                                   family = family,
                                   a0_seq = a0_seq,
                                   c0 = c0,
                                   d0 = d0,
                                   delta0 = delta0,
                                   lambda0 = lambda0,
                                   iter_warmup = iter_warmup,
                                   iter_sampling = iter_sampling,
                                   num_cores = 10)

post_samples_trt <- samples_models(data = data_trt,
                                   outcome = "outcome",
                                   family = family,
                                   a0_seq = a0_seq,
                                   c0 = c0,
                                   d0 = d0,
                                   delta0 = delta0,
                                   lambda0 = lambda0,
                                   iter_warmup = iter_warmup,
                                   iter_sampling = iter_sampling,
                                   num_cores = 10)

```

Now, we create auxiliary functions to compute the average treatment effect:

```

predict_ <- function(data, outcome, trt, family, beta_post){
  # get covariates
  cov_data <- colnames(data)[!(colnames(data) %in% c(outcome, trt))]
  # compute product of beta and covariates
  prod <- lapply(beta_post,
                 function(post_samp) {
                   cov <- names(post_samp)[names(post_samp) %in% cov_data]
                   post_samp <- as.data.frame(post_samp[c("(Intercept)", cov)])
                   cov_mat <- cbind(1, data.matrix(data[cov]))
                   cov_mat <- t(cov_mat)
                   beta <- data.matrix(post_samp)
                   beta %*% cov_mat
                 })
  )
  # compute predictions
  if (family$link == "logit") {
    pred <- lapply(prod, function(df) {
      apply(df, 2, plogis)
    })
  } else if (family$link == "probit") {
    pred <- lapply(prod, function(df) {
      apply(df, 2, pnorm)
    })
  }
  return(pred)
}

```

```

# function to compute average effect of each model and group
mean_models_arm <- function(data, outcome, trt, family, beta_post){
  # compute predictions
  pred <- predict_(data, outcome, trt, family, beta_post)
  # compute means for each model
  mean_models <- lapply(pred,
    function(x) {
      apply(x, 1, function(row){
        sample_weights <- t(rdirichlet(1, rep(1, nrow(data))))
        row %*% sample_weights
      })
    })
  return(do.call(cbind, mean_models))
}

# function to compute average effect of each group
mean_arm <- function(post_models, mean_arm_models){
  mean_arm_models %*% post_models
}

```

We can observe the results:

```

post_samples_trt$df_post_ord
post_samples_ctrl$df_post_ord

#> post_samples_trt$df_post_ord
#>      model prior_model      ml post_model
#> 1      cd4 2.162312e-01 2.484787e-06 3.717036e-01
#> 2      age, cd4 4.273791e-01 1.150383e-06 3.401301e-01
#> 3 age, race, cd4 2.426728e-01 8.802853e-07 1.477862e-01
#> 4      race, cd4 1.135210e-01 1.787428e-06 1.403764e-01
#> 5      age 1.097845e-04 2.851294e-08 2.165570e-06
#> 6      age, race 6.291359e-05 2.168054e-08 9.436347e-07
#> 7      race 2.317249e-05 3.882745e-08 6.224450e-07

#> post_samples_ctrl$df_post_ord
#>      model prior_model      ml post_model
#> 1      age, cd4 4.280434e-01 5.416766e-10 4.723140e-01
#> 2 age, race, cd4 2.419794e-01 6.638414e-10 3.272243e-01
#> 3      cd4 2.165200e-01 2.666885e-10 1.176265e-01
#> 4      race, cd4 1.132612e-01 3.589752e-10 8.282250e-02
#> 5      age 1.099827e-04 3.149964e-11 7.057207e-06
#> 6      age, race 6.285405e-05 3.853906e-11 4.934433e-06
#> 7      race 2.323827e-05 1.686888e-11 7.985330e-07

```

Finally, we compute the average treatment effect:

```

mean_models_ctrl <- mean_models_arm(current_data_ctrl,
  "outcome",
  "treatment",
  family,
  post_samples_ctrl$post_betam)

```



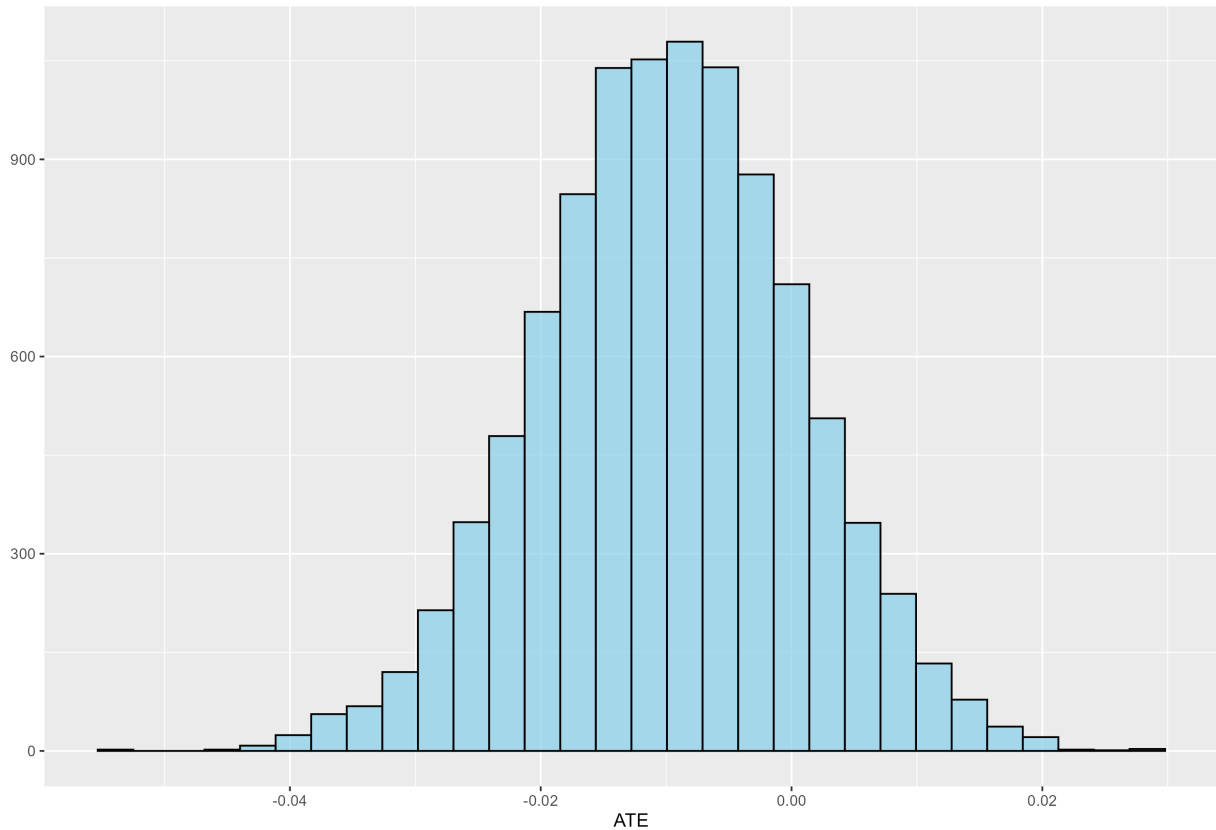
```

mean_models_trt <- mean_models_arm(current_data_trt,
                                   "outcome",
                                   "treatment",
                                   family,
                                   post_samples_trt$post_betam)
mean_ctrl <- mean_arm(post_samples_ctrl$df_post$post_model, mean_models_ctrl)
mean_trt <- mean_arm(post_samples_trt$df_post$post_model, mean_models_trt)

ate <- mean_trt - mean_ctrl

```

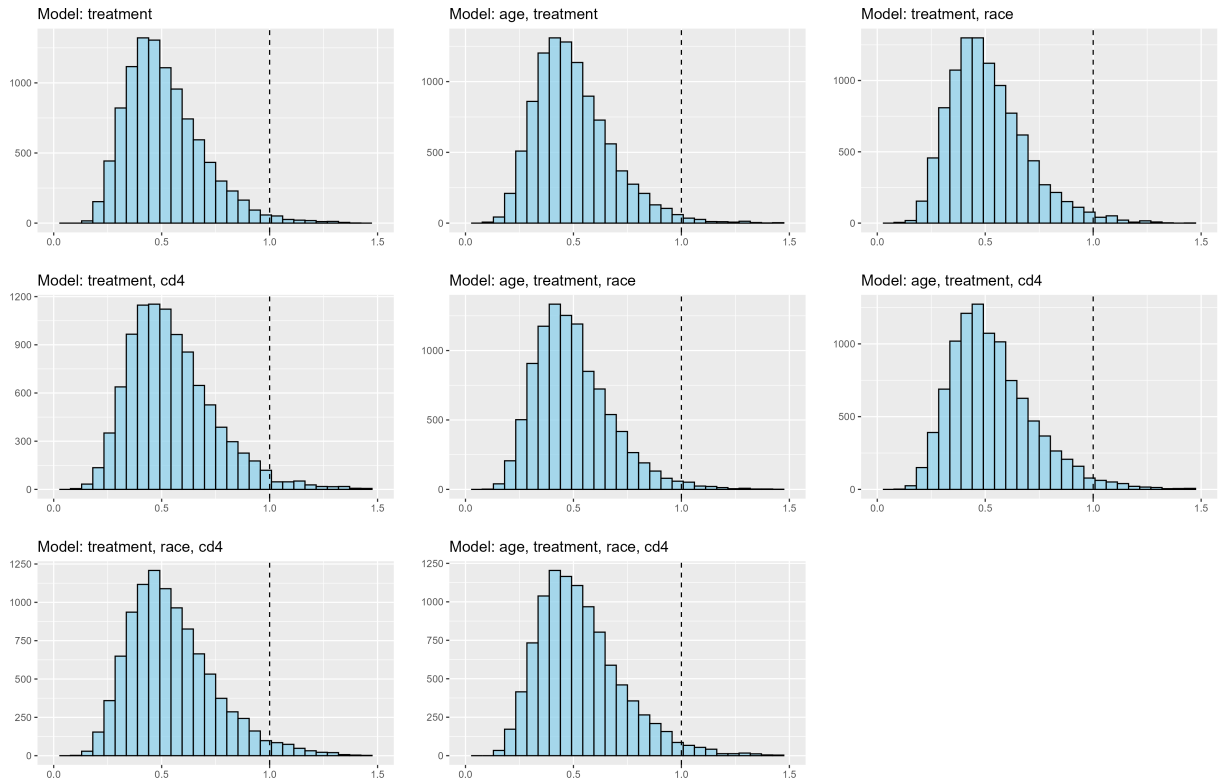
Posterior distribution of the average treatment effect (ATE)



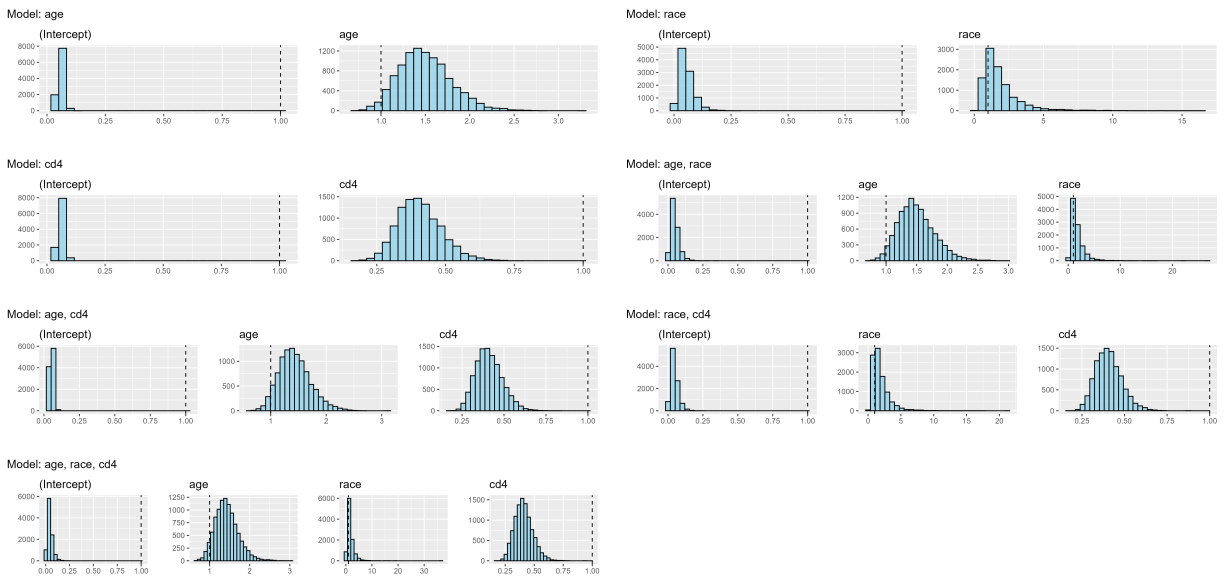
Intermediate results

We can analyze the posterior distribution of the odds ratio in the complete data set:

Posterior distribution of the odds ratio of treatment effect



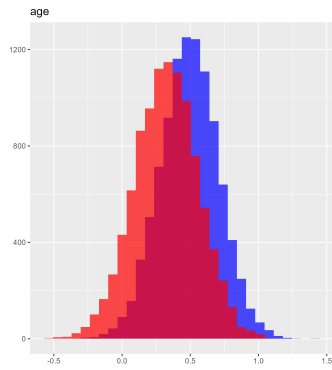
Posterior distribution of the odds ratio (models without treatment effect)



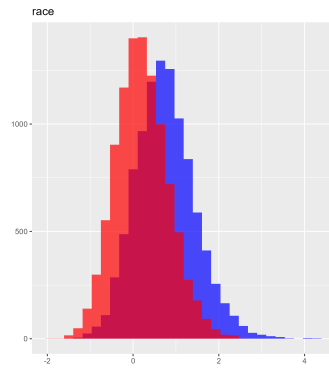
And the posterior distribution of covariate effects by arm:

Posterior distribution of covariate effects by arm - Part 1

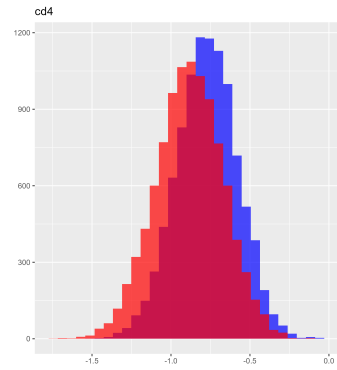
Model: age



Model: race

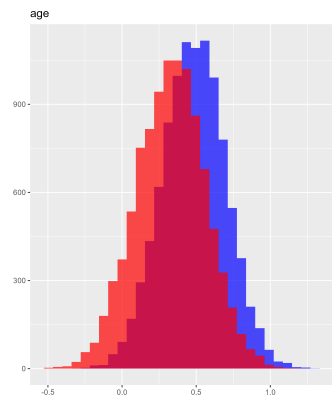


Model: cd4

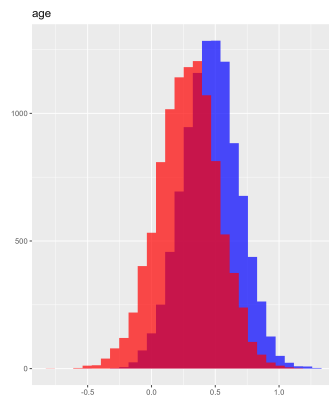


Posterior distribution of covariate effects by arm - Part 2

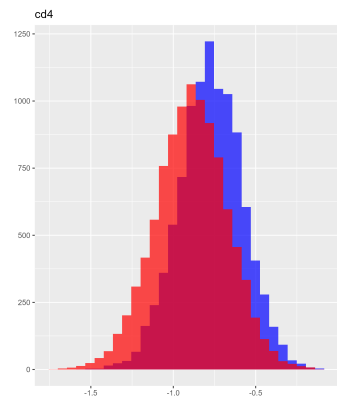
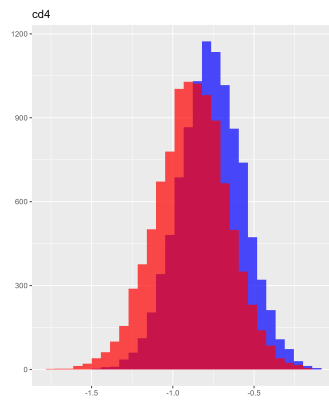
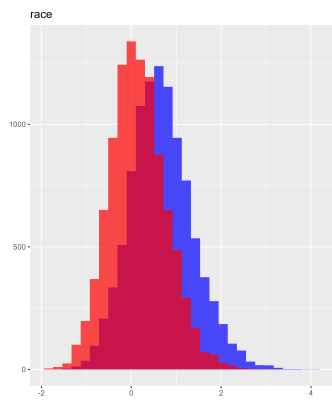
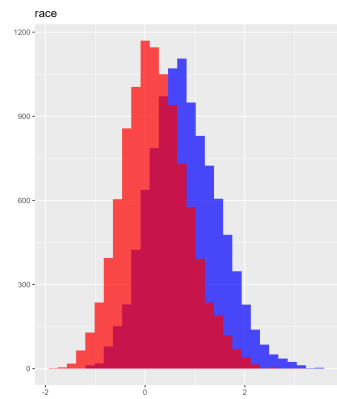
Model: age, race



Model: age, cd4

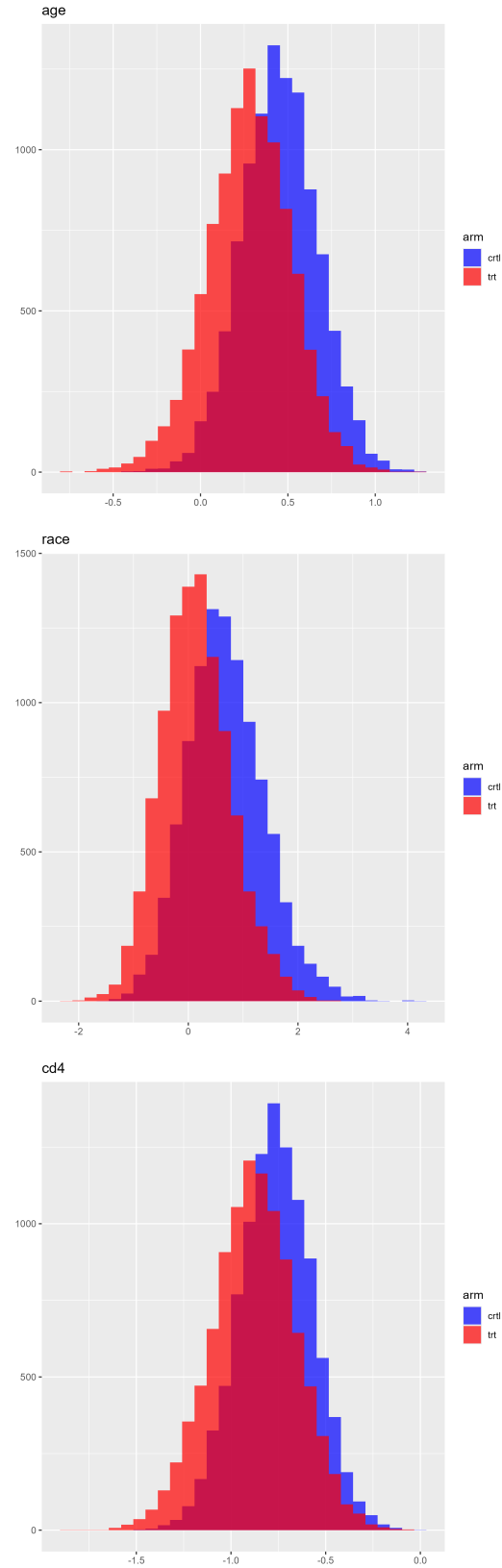


Model: race, cd4



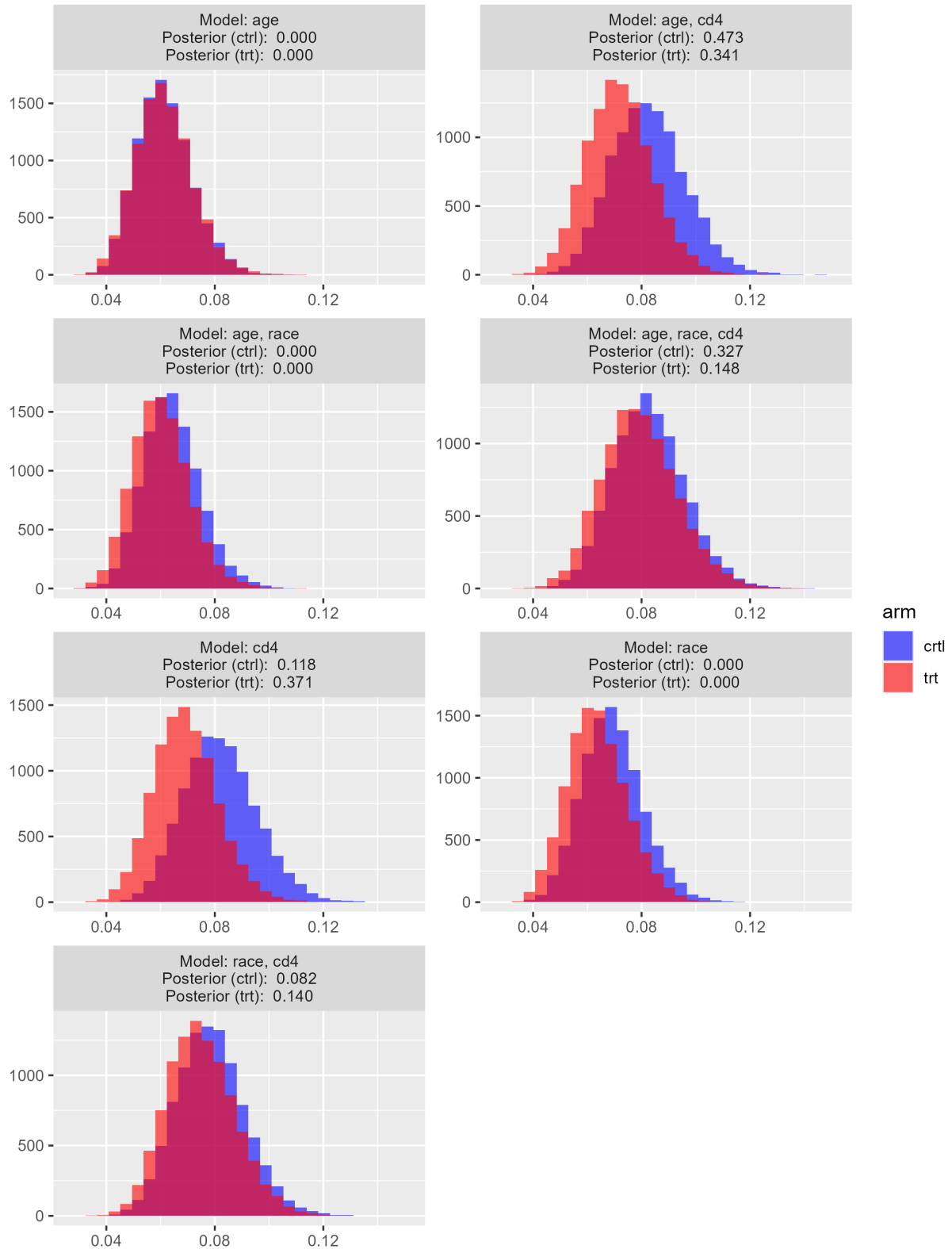
Posterior distribution of covariate effects by arm - Part 3

Model: age, race, cd4



Lastly, we can observe the distribution of marginal means by arm:

Posterior distribution of marginal means by arm



Another computation - Bayesian model averaging

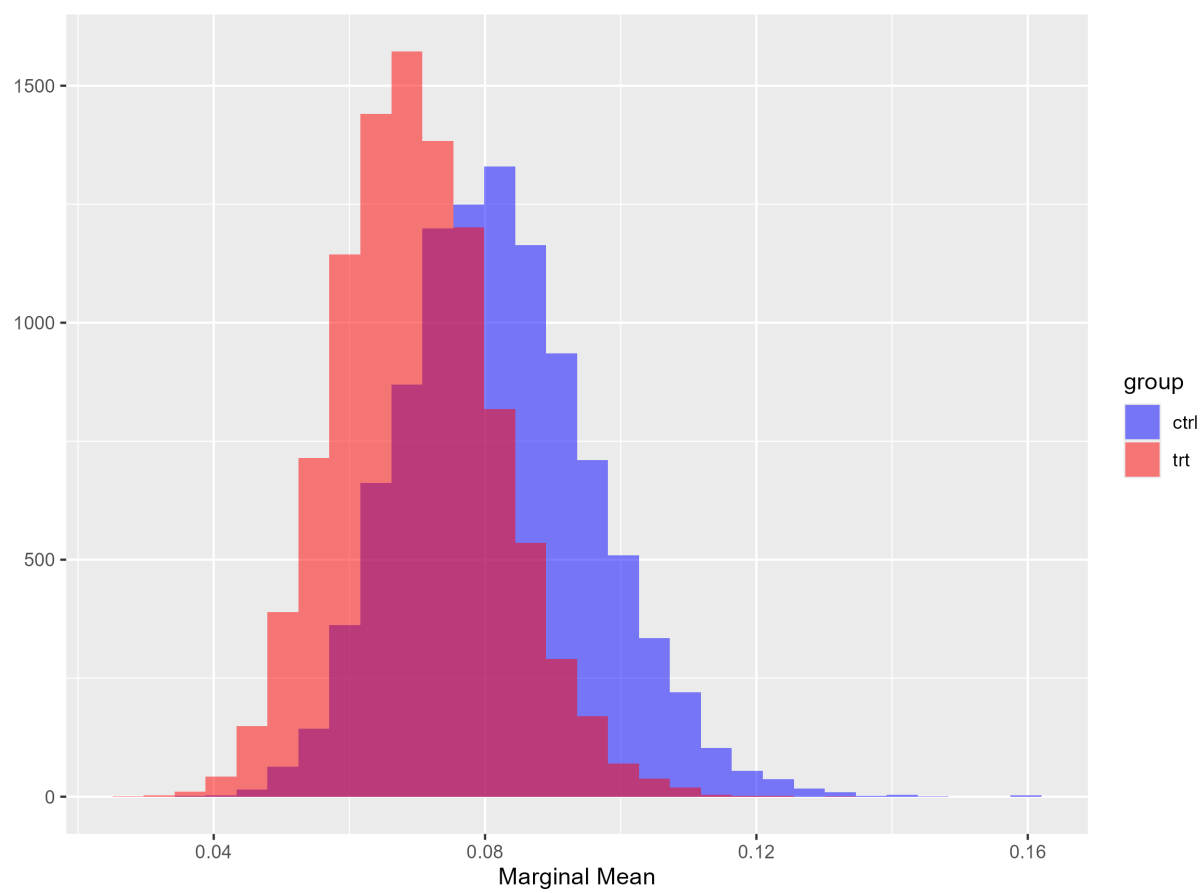
We can also compute the average treatment effect using Bayesian model averaging. The algorithm is as follows:

1. Compute the marginal means for each model and group.
2. For each group:
 1. Sample an index k from the posterior distribution of the models.
 2. Sample a draw from the posterior distribution of the marginal means for model k .
 3. Repeat the previous step N times.

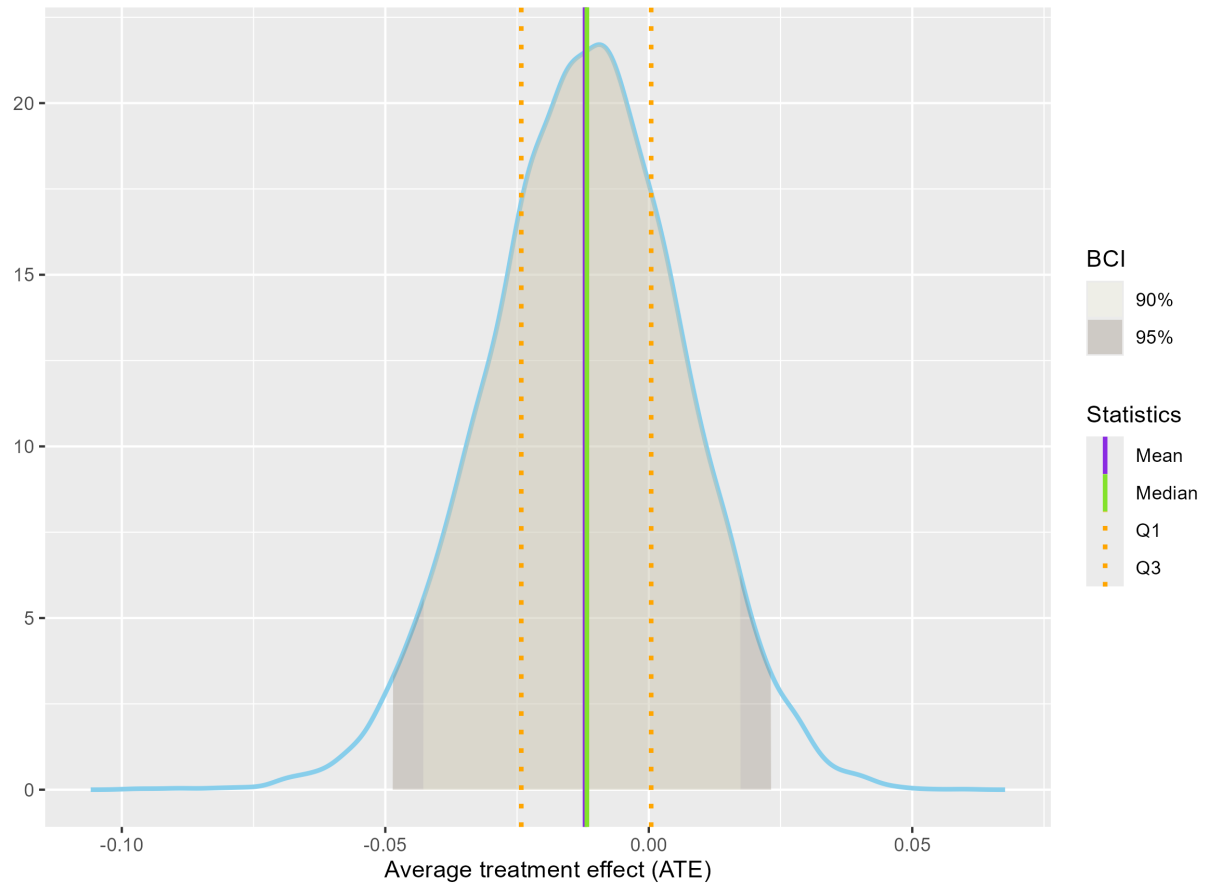
The code is as follows:

```
bma <- function(x, df_post, n_samples = 1000){  
  # sample models  
  ks <- sample(1:nrow(df_post), n_samples, replace = TRUE, prob = df_post$post_model)  
  # sample marginal means  
  samples <- lapply(ks, function(k) {  
    id_draw <- sample(1:nrow(x), 1)  
    draw_x <- x[id_draw,k]  
  })  
  return(do.call(rbind, samples))  
}
```

From the samples, we can compute the marginal means for each group:



And the average treatment effect:



Finally, we can summarize the results:

statistic	value
Mean	-0.012
Median	-0.012
Q1	-0.024
Q3	0.000
90% BCI	[-0.043, 0.018]
95% BCI	[-0.049, 0.023]

Table 1: BMA summary (ATE)